Project: To-Do List API **CR ID:** CR-002 — API Idempotency

1. Technical Objective

Prevent duplicate `Task` creation caused by client retries after a lost response, by persisting a record of each idempotency-key-tagged request and its outcome, and replaying that outcome on a matching retry instead of re-executing the creation.

2. Proposed Solution

Introduce a new `IDEMPOTENCY_KEY` table (and corresponding `IdempotencyRecord` entity) that stores: the client-supplied key, a fingerprint (hash) of the request payload, the resulting task id, the stored response body, and a creation timestamp. `TaskServiceImpl.create()` is wrapped by a check against this table before any `Task` row is written: new key → proceed and record; known key + matching fingerprint → return the stored response; known key + different fingerprint → `409 Conflict`.

A unique database constraint on the key column is the real concurrency safeguard — the two-concurrent-requests case (FR-06) is resolved by letting both attempt to insert the idempotency record first; the one that loses the unique-constraint race re-reads the winner's stored response instead of proceeding to create a task.

3. Components to Modify

Component	File	Change
Task controller	`controller/TaskController.java`	Read optional `Idempotency-Key` header, pass to service
Task service	`service/TaskService.java`, `service/impl/TaskServiceImpl.java`	`create()` gains idempotency-check wrapper logic
Error handling	`exception/GlobalExceptionHandler.java`	Handler for key-reuse-with-different-payload → `409`
`application.yml`	config	New property for idempotency record retention period

New components:

Component	Purpose
`entity/IdempotencyRecord.java`	New entity mapping the idempotency table
`repository/IdempotencyRecordRepository.java`	`findByIdempotencyKey(String key)`, standard `JpaRepository`
`exception/IdempotencyKeyConflictException.java`	Thrown on fingerprint mismatch for a reused key
A scheduled cleanup (reuses the same scheduling mechanism proposed in CR-003) to purge expired records	Addresses FR-07

4. Database Changes (JPA/Hibernate)

New table, mapped via JPA (no manual SQL):

```
@Entity
@Table(name = "IDEMPOTENCY_RECORD")
public class IdempotencyRecord {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;
```

```
        @Column(name = "IDEMPOTENCY_KEY", nullable = false, unique = true, length = 200)
        private String idempotencyKey;

        @Column(name = "REQUEST_FINGERPRINT", nullable = false, length = 64)
        private String requestFingerprint; // SHA-256 hash of the normalized request body

        @Column(name = "RESPONSE_BODY", nullable = false)
        @Lob
        private String responseBody; // stored JSON of the original TaskResponse

        @Column(name = "RESPONSE_STATUS", nullable = false)
        private int responseStatus; // normally 201

        @Column(name = "CREATED_AT", nullable = false)
        private Instant createdAt;
    }
```

- `idempotencyKey` carries a `unique = true` constraint — this is what makes FR-06 (concurrent same-key requests) safe: the loser of the race gets a `DataIntegrityViolationException` on insert, which the service layer catches and turns into "re-read and return the winner's stored response" rather than an error.
- **Migration consideration:** this is a brand-new table, so there is no existing-data migration risk (unlike CR-001). `ddl-auto: update` is sufficient to create it.
- Cleanup (FR-07) is a `DELETE FROM IDEMPOTENCY_RECORD WHERE CREATED_AT < :cutoff`, run on a schedule — see CR-003 for the scheduling mechanism this CR reuses; the retention window itself is a new `application.yml` property (e.g. `app.idempotency.retention-hours`), proposed default 24 hours.

5. API Design

No new endpoints. `POST /api/tasks` gains an optional request header.

Example — first request

```
POST /api/tasks
Idempotency-Key: 8f14e45f-ceea-467e-bd97-0123456789ab

{
  "title": "Prepare report",
  "categoryId": 3
}
```

Response `201 Created` — identical shape to today's `TaskResponse`.

Example — retry with the same key and body

```
POST /api/tasks
Idempotency-Key: 8f14e45f-ceea-467e-bd97-0123456789ab

{
  "title": "Prepare report",
  "categoryId": 3
}
```

Response: the **same** `201 Created` body as the first request (same task `id`), no new row created.

Example — key reused with a different body

```
POST /api/tasks
Idempotency-Key: 8f14e45f-ceea-467e-bd97-0123456789ab

{
  "title": "Different task entirely",
  "categoryId": 1
```

```
}
```

Response `409 Conflict`:

```
{
  "timestamp": "2026-10-15T10:30:00Z",
  "status": 409,
  "error": "Conflict",
  "message": "Idempotency-Key '8f14e45f-...' was already used with a different request body",
  "path": "/api/tasks"
}
```

6. Processing Logic

1. `TaskController.create()` reads the `Idempotency-Key` header, if present, and passes it (alongside the existing `TaskRequest`) to `TaskServiceImpl.create()`.
2. If no key is present → proceed exactly as today (FR-05); skip steps 3–6.
3. Compute `fingerprint = SHA-256(normalized request JSON)`.
4. Attempt to **insert** a new `IdempotencyRecord` with `(key, fingerprint, status=PENDING-equivalent)` inside the same transaction as the task creation that follows.
    - If the insert succeeds (key is new): proceed to create the `Task` as normal, then update the idempotency record with the resulting `responseBody` / `responseStatus`, and commit both as one transaction.
    - If the insert fails on the unique constraint (key already exists): re-read the existing record by key.
        - If its `requestFingerprint` matches → return its stored `responseBody` / `responseStatus` (FR-03). No task is created.
        - If it does not match → throw `IdempotencyKeyConflictException` (FR-04), mapped to `409`.
5. For the concurrent case (FR-06): both requests attempt the insert; the database's unique constraint guarantees only one succeeds. The loser follows the "re-read and compare fingerprint" path above — if the winner's transaction hasn't committed yet, the loser's read may briefly see a pending record; the design accepts a short retry/backoff in that narrow window rather than introducing additional locking.
6. `TaskController` returns the resulting `ResponseEntity` exactly as it does today — the idempotency layer is transparent to the response shape.

7. Error Handling

Scenario	HTTP Status	Notes
No `Idempotency-Key` header	`201 Created` (normal flow)	Unchanged existing behavior
New key, request valid	`201 Created`	Normal flow, now also recorded
Known key, matching payload	`201 Created` (replayed)	Original response returned verbatim; no new row
Known key, mismatched payload	`409 Conflict`	New `IdempotencyKeyConflictException` handler
Request body fails existing validation (`@Valid`)	`400 Bad Request`	Unchanged — validation runs before idempotency logic

8. Test Scenarios

See `TEST_CASES.md`. Summary: first use of a key creates one task and records it; replay with the same key/body returns the same task without a new row; replay with the same key/different body returns `409`; omitted key behaves as today; concurrent identical requests under the same new key yield exactly one created task.

9. Technical Decision

Selected approach: Persisted idempotency records in Oracle, keyed by a client-supplied `Idempotency-Key` header, with a unique constraint as the concurrency safeguard.

Reason for selection: Reuses the project's existing persistence layer (Oracle via Spring Data JPA) with no new infrastructure, matches the project's existing pattern of using a unique database constraint as a safety net (already used for `Category.name`), and keeps idempotency records durable across application restarts — important since the whole point is surviving a lost response, which may span a restart.

Alternatives considered:

- **In-memory cache (e.g. a `ConcurrentHashMap` or Caffeine cache) of recent keys:** rejected — lost on restart, and would not survive the project potentially running multiple instances behind a load balancer in the future.
- **Hashing the entire request as the sole key (no explicit header):** rejected — this prevents a client from legitimately creating two *identical* tasks on purpose (e.g. two separate "Follow up with client" tasks), which an explicit idempotency key avoids by scoping "sameness" to a single retry sequence rather than to payload content alone.
- **Redis or another external cache for idempotency keys:** rejected as unnecessary infrastructure for this project's current scale, per the project's architecture rules (avoid premature infrastructure).

10. Technical Approval

- **Prepared By:** Backend Developer
- **Reviewed By:** Tech Lead
- **Approval Status:** Pending
- **Implementation Status:** Not Started